Space lower bounds for adversarially robust missing-item algorithms using a random tape

Amit Chakrabarti, Manuel Stoeckl

doi:10.4230/LIPIcs.CCC.2024.28; CC BY-SA4.0

Editorial note. Original source: Amit Chakrabarti and Manuel Stoeckl, arXiv:2310.03634v2 (2024-07-03), https://arxiv.org/abs/2310.03634v2. License: Creative Commons Attribution-ShareAlike4.0, https://creativecommons.org/licenses/by-sa/4.0/.

Editorial note. Changes: selected complete author theorem/proof/context excerpts and original diagram/macros/bibliography in a standalone layout; omitted unrelated sections and author frontmatter. No mathematical corrections or additional proofs. This adapted presentation retains CC BY-SA4.0.

Allowed standard prerequisites (editorial).

- One-way avoid communication lower bound. Author Theorem lem:avoid-lb explicitly cites ChakrabartiGS22; complete quoted statement and problem definition retained.
- Multiplicative Chernoff bounds under negative association. Author Lemma lem:chernoff-neg-assoc explicitly cites JoagDevP83; statement retained.

Source reference. Original section 2.2: number-theory/sources/ccc-2024-28/native/main-content.tex, source lines 615–615. Introductory overview pointer only; complete adversary and all proof details are included.

Source reference. Original reference 2: number-theory/sources/ccc-2024-28/native/main-content.tex, source lines 267–267. Introductory weakened summary is a consequence of the delivered exact theorem; it is not used as a premise.

In this paper, we prove that for certain parameter regimes, MissingItemFinding (henceforth, MIF) is such a problem. In the problem MIF(n, ℓ), the input is a stream $\langle e_1, \ldots, e_\ell \rangle$ of ℓ integers, not necessarily distinct, with each $e_i \in \{1, \ldots, n\}$, where $1 \leq \ell \leq n$. The goal is as follows: having received the ith integer, output a number v in $\{1, \ldots, n\} \setminus \{e_1, \ldots, e_i\}$. We will be mostly interested in the setting $\ell = o(n)$, so the "trivial" upper bound on the space complexity of MIF(n, ℓ) is $O(\ell \log n)$, achieved by the deterministic algorithm that simply stores the input stream as is.

0.1 Groundwork for Our Results

To state our results about MIF, we need to introduce some key terminology. Notice that MIF is a *tracking problem*: an output is required after reading each

input.[1] Thus, we view streaming algorithms as generalizations of finite state (Moore-type) machines. An algorithm $\mathcal{A}$ has a finite set of states Σ (leading to a space cost of $\log_2 |\Sigma|$), a finite input set $\mathcal{I}$, and a finite output set $\mathcal{O}$. It has a transition function $T\colon \Sigma \times \mathcal{I} \times \mathcal{R} \to \Sigma$ indicating the state to switch to after receiving an input, plus an output function $\gamma\colon \Sigma \times \mathcal{R} \to \mathcal{O}$ indicating the output produced upon reaching a state. How the final parameter (in $\mathcal{R}$) of T and γ is used depends on the type of randomness. We consider four cases, leading to four different models of streaming computation.

- *Deterministic.* The initial state of the algorithm is a fixed element of Σ, and T and γ are deterministic (they do not depend on the parameter in $\mathcal{R}$).
- *Random seed.* The initial state is drawn from a distribution $\mathcal{D}$ over Σ, and T and γ are deterministic. This models the situation that all random bits used count towards the algorithm's space cost.
- *Random tape.* The initial state is drawn from a distribution $\mathcal{D}$ over Σ.[2] The space $\mathcal{R}$ is a sample space; when the algorithm receives an input $e \in \mathcal{I}$ and is at state $\sigma \in \Sigma$, it chooses a random $\rho \in \mathcal{R}$ independent of all previous choices and moves to state $T(e, \sigma, \rho)$. However, γ is deterministic.[3] This models the situation that the algorithm can make random decisions at any time, but it cannot remember past random decisions without recording them (which would add to its space cost).
- *Random oracle.* The initial state is fixed; $\mathcal{R}$ is a sample space. A specific $R \in \mathcal{R}$ is drawn at the start of the algorithm and stays the same over its lifetime. When the algorithm is at state σ and receives input e, its next state is $T(e, \sigma, R)$. The output given at state σ is $\gamma(\sigma, R)$. This models the situation that random bits are essentially "free" to the algorithm; it can read from a long random string which doesn't count toward its space cost and which remains consistent over its lifetime. A random oracle algorithm can be interpreted as choosing a random deterministic algorithm, indexed by R, from some family.

These models form a rough hierarchy; they have been presented in (almost) increasing order of power. Every z-bit (2^z-state) deterministic algorithm can be implemented in any of the random models using z bits of space; the same holds for any z-bit random seed algorithm. Every z-bit random tape algorithm has a corresponding $(z + \log \ell)$-bit random oracle algorithm—the added space cost is because for a random oracle algorithm to emulate a random tape algorithm, it must have a way to get "fresh" randomness on each turn.[4]

[1] We do not consider algorithms with a "one-shot" guarantee, to only be correct at the end of the stream, because a) the adversarial setting requires tracking output b) for MIF and most other problems the difference in space complexity is generally small.

[2] Requiring that this model use a fixed initial state could make some algorithms use one additional "INIT" state.

[3] Alternatively, we could associate a *distribution* of outputs to each state, or a function mapping (input, state) pairs to outputs. As these formulations are slightly more complicated to prove things with, and only affect the space usage of MISSINGITEMFINDING algorithms by an additive $O(\log n + \log \frac{1}{\delta})$ amount, we stick with the one state = one output convention.

[4] An alternative, which lets one express z-bit random tape algorithms using a z-bit random oracle variant, is to assume the random oracle algorithm has access to a clock or knows the position in the stream for free; both are reasonable assumptions in practice.

Streaming algorithms are also classified by the kind of correctness guarantee they provide. Recall that we focus on "tracking" algorithms [BJWY20]; they present an output after reading each input item and this *entire sequence* of outputs must be correct. Here are three possible meanings of the statement "algorithm $\mathcal{A}$ is δ-error" (we assume that $\mathcal{A}$ handles streams of length ℓ with elements in $\mathcal{I}$ and has outputs in $\mathcal{O}$):

- *Static setting.* For all inputs $\tau \in \mathcal{I}^\ell$, running $\mathcal{A}$ on τ produces incorrect output with probability $\leq \delta$.
- *Adversarial setting.* For all (computationally unbounded) adaptive adversaries α (i.e., for all functions $\alpha\colon \mathcal{O}^\star \to \mathcal{I}$),[5] running $\mathcal{A}$ against α will produce incorrect output with probability $\leq \delta$.
- *Pseudo-deterministic setting.* There exists a canonical output function $f\colon \mathcal{I}^\star \to \mathcal{O}$ producing all correct outputs so that, for each $\tau \in \mathcal{I}^\ell$, $\mathcal{A}(\tau)$ fails to output $f(\tau)$ with probability $\leq \delta$.

Algorithms for the static setting are called "classic" streaming algorithms; ones for the adversarial setting are called "adversarially robust" streaming algorithms. All pseudo-deterministic algorithms are adversarially robust, and all adversarially robust algorithms are also classic.

As a consequence of Newman's theorem [New91], any random oracle or random tape algorithm in the static setting with error δ can be emulated using a random seed algorithm with only ε increase in error and an additional $O(\log \ell + \log\log |\mathcal{I}| + \log \frac{1}{\varepsilon\delta})$ bits of space. However, the resulting algorithm is non-constructive.

1 Preliminaries

Notation. Throughout this paper, $\log x = \log_2 x$, while $\ln x = \log_e x$. The set $\mathbb{N}$ consists of all positive integers; $[k] := \{1, 2, \ldots, k\}$; and $[a, b)$ is a half open interval of real numbers. For a condition or event E, the symbol $\mathbb{1}_E$ takes the value 1 if E occurs and 0 otherwise. The sequence (stream) obtained by concatenating sequences a and b, in that order, is denoted $a \circ b$. For a set S of elements in a totally ordered universe, $\textsc{sort}(S)$ denotes the sequence of elements of S in increasing order; $\binom{S}{k}$ is the set of k-element subsets of S; and $\textsc{seqs}(S, k) = \{\textsc{sort}(Y) : Y \in \binom{S}{k}\}$. We sometimes extend set-theoretic notation to vectors and sequences; e.g., for $y \in [n]^t$, write $y \subseteq S$ to mean that $\forall i \in [t] : y_i \in S$. For a set X, $\triangle[X]$ denotes the set of probability distributions over X, while $A \in_R X$ indicates that A is chosen uniformly at random from X. When naming probability distributions, we will use either calligraphic letters (e.g., $\mathcal{A}, \mathcal{D}$) or the letter μ; $A \sim \mu$ means that A is drawn from the distribution μ.

As outlined in Original section 2.2, $\textsc{avoid}(m, a, b)$ is a one-way communication problem, wherein player Alice has a set $A \in \binom{[m]}{a}$, and should send a short message to player Bob, who should use the message to output a set $B \in \binom{[m]}{b}$ that is disjoint from A. In the randomized δ-error setting, this disjointness should hold with probability $\geq 1 - \delta$.

[5] By the minimax theorem, it suffices to consider deterministic adversaries.

Theorem 1.1 (AVOID communication lower bound, from [CGS22])**.** *Suppose there exists a randomized protocol for* AVOID(m, a, b)*, in which Alice communicates $\leq K$ bits, that is δ-error either on a worst-case input or when Alice's input is chosen uniformly at random from $\binom{[t]}{a}$. Then*

$$K \geq \frac{ab}{t \ln 2} + \log(1 - \delta)\,.$$

Using Theorem 1.1, it is straightforward to derive the following lower bound for robust algorithms for MIF(n, ℓ), as was done in [Sto23]. The proof is short, yet instructive, so we outline it here.

Theorem 1.2 (Adversarially robust random oracle lower bound, from [Sto23])**.** *If there exists a δ-error adversarially robust random oracle algorithm for* MIF(n, ℓ) *using z bits of space, then*

$$z \geq \frac{\ell^2}{4n \ln 2} + \log(1 - \delta)\,.$$

Lemma 1.3 (Multiplicative Azuma's inequality)**.** *Let $X_1, \ldots, X_t$ be $[0, 1]$ random variables, and $\alpha \geq 0$. If, for all $i \in [t]$, $\mathbb{E}[X_i \mid X_1, \ldots, X_{i-1}] \leq p_i$, then*

$$\Pr\left[\sum_{i=1}^{t} X_i \geq (1+\alpha)\sum_{i=1}^{t} p_i\right] \leq \exp\left(-((1+\alpha)\ln(1+\alpha) - \alpha)\sum_{i=1}^{t} p_i\right) \leq \exp\left(-\frac{\alpha^2}{2+\alpha}\sum_{i=1}^{t} p_i\right)\,.$$

On the other hand, if for all i, $\mathbb{E}[X_i \mid X_1, \ldots, X_{i-1}] \geq p_i$, then

$$\Pr\left[\sum_{i=1}^{t} X_i \leq (1-\alpha)\sum_{i=1}^{t} p_i\right] \leq \exp\left(-((1-\alpha)\ln(1-\alpha) + \alpha)\sum_{i=1}^{t} p_i\right) \leq \exp\left(-\frac{\alpha^2}{2}\sum_{i=1}^{t} p_i\right)\,.$$

In contrast to the above, the "usual" form of Azuma's inequality uses a martingale presentation and gives an additive-type bound.

Lemma 1.4 (Chernoff bound with negative association, from [JP83])**.** *The standard multiplicative Chernoff bounds work with negatively associated random variables.*

Proof of Theorem 1.3. The proof is modeled off that in [Sto23], which only addresses one direction. It is a straightforward blend of standard proofs of the Chernoff bound and of Azuma's inequality.

First, the $\geq$ direction. Choose, with foresight, $z = \ln(1 + \alpha)$.

$$\begin{aligned}
&\Pr\left[\sum_{i=1}^{t} X_i \geq (1+\alpha)\sum_{i=1}^{t} p_i\right] \\
&= \Pr\left[\exp(z\sum_{i=1}^{t} X_i) \geq \exp(z(1+\alpha)\sum_{i=1}^{t} p_i)\right] \\
&\leq \frac{\mathbb{E}\exp(z\sum_{i=1}^{t} X_i)}{\exp(z(1+\alpha)\sum_{i=1}^{t} p_i)} \\
&\leq \frac{\mathbb{E}[e^{zX_1}\mathbb{E}[e^{zX_2}\ldots\mathbb{E}[e^{zX_t}|X_1,\ldots,X_{t-1}]\ldots|X_1]]}{\exp(z(1+\alpha)\sum_{i=1}^{t} p_i)}\,.
\end{aligned}$$

The innermost term $\mathbb{E}[e^{zX_t}|X_1,\ldots,X_{t-1}]$ is, by convexity of e^z, $\leq p_t e^z + (1-p_t) \leq e^{p_t(e^z-1)}$; after applying this upper bound, we can factor it out and bound the X_{t-1} term, and so on. Thus we continue the chain of inequalities to get:

$$\leq \frac{\exp\left((e^z-1)\sum_{i=1}^t p_i\right)}{\exp\left(z(1+\alpha)\sum_{i=1}^t p_i\right)} = \exp\left(-((1+\alpha)\ln(1+\alpha)-\alpha)\sum_{i=1}^t p_i\right).$$

For the other direction, set $z = \ln(1-\alpha)$, which is < 0. This time, $\mathbb{E}[e^{zX_t}|X_1,\ldots,X_{t-1}] \leq p_t e^z + (1-p_t)$ because p_t is a *lower bound* for $\mathbb{E}[X_t|X_1,\ldots,X_{t-1}]$, and z is negative. That $p_t e^z + (1-p_t) \leq e^{p_t(e^z-1)}$ still holds for negative z, so:

$$\begin{aligned}\Pr\left[\sum_{i=1}^t X_i \leq (1-\alpha)\sum_{i=1}^t p_i\right] &= \Pr\left[\exp(z\sum_{i=1}^t X_i) \geq \exp(z(1-\alpha)\sum_{i=1}^t p_i)\right] \\ &\leq \ldots \leq \frac{\exp\left((e^z-1)\sum_{i=1}^t p_i\right)}{\exp\left(z(1-\alpha)\sum_{i=1}^t p_i\right)} \\ &= \exp\left(-((1-\alpha)\ln(1-\alpha)+\alpha)\sum_{i=1}^t p_i\right). \quad \square\end{aligned}$$

Proof. Such an algorithm $\mathcal{A}$ yields a z-bit δ-error randomized protocol for AVOID$(n, \lceil \ell/2 \rceil, \lfloor \ell/2 \rfloor + 1)$ wherein Alice feeds her set A into $\mathcal{A}$, sends the state of $\mathcal{A}$ to Bob, and Bob extracts set B by using the "echo" adversarial strategy, i.e., repeatedly asking $\mathcal{A}$ for an output item and feeding that item back as the next input. The result now follows by appealing to Theorem 1.1. $\square$

We also note the following simple lower bound.

Lemma 1.5. *For every $\delta < 1$, if there exists a δ-error random tape algorithm for MIF(n,ℓ) using z bits of space, then $z \geq \log(\ell+1)$.*

Proof. Each state of a random tape streaming algorithm $\mathcal{A}$ has a unique associated output value. If $z < \log(\ell+1)$, then $\mathcal{A}$ has at most ℓ states. Let H be the set of outputs associated with these states; so $|H| \leq \ell$. When sent a stream containing each element of H, $\mathcal{A}$ will fail with probability 1 because every output it could make is wrong. $\square$

By the remarks in Section 0.1 following the definitions of the models of computation, Theorem 1.2 also applies to random tape and random seed algorithms and Theorem 1.5 also applies to random seed algorithms.

2 The Random Tape Lower Bound

This section presents our first and perhaps most important lower bound, of which Original reference 2 is a consequence. We shall carry out the proof plan outlined in Original section 2.2, designing a recursive adversary to foil a given random-tape MIF algorithm $\mathcal{A}$ that runs in z bits of space. Correspondingly, our lower bound proof will be inductive.

2.1 Setup and Base Case

Recall that the adversary organizes the ℓ-length input to be fed into $\mathcal{A}$ as a random prefix of length $\ell/2$ followed by another $\ell/2$ inputs divided into several phases, each consisting of $t = O(\ell/z)$ inputs. The adversary's eventual goal is to identify a particular phase and a corresponding sub-universe $W \subseteq [n]$ so that $\mathcal{A}$, when suitably conditioned and restricted to that phase, yields a sub-algorithm $\mathcal{B}$ that is good for MIF for inputs from W. However, we will need to generalize the notion of a "good" MIF algorithm, because this sub-algorithm might produce outputs outside of W, even when fed inputs from W. To aid our analysis, we will make $\mathcal{B}$ abort anytime it would have produced an output outside of W. In what follows, it will be important to maintain a distinction between these aborts and actual mistakes.

Definition 2.1. An algorithm $\mathcal{A}$ for $\textsc{mif}(n, \ell)$ can fail in either of two ways. It may make an incorrect output, or *mistake*, if outputs an element in $[n]$ that *is* in its input stream (i.e., not missing). It may also *abort*, by outputting a special value $\perp$ (where $\perp \notin [n]$) and stopping its run.

For integers n, ℓ, z with $1 \le \ell < n$, and $\gamma \in [0, 1]$, let $\textsc{Algs}(n, \ell, \gamma, z)$ be the set of all z-bit *random tape* algorithms for $\textsc{mif}(n, \ell)$ which on *any* adversary abort with probability $\le \gamma$. Define

$$\Delta(n, \ell, \gamma, z) := \min_{\mathcal{A} \in \textsc{Algs}(n,\ell,\gamma,z)} \delta_{\max}(\mathcal{A}, n, \ell)\,,$$

where $\delta_{\max}(\mathcal{A}, n, \ell)$ is the maximum probability, over all possible adversaries, that $\mathcal{A}$ makes a mistake. As a consequence of the definition, $\Delta(n, \ell, \gamma, z)$ is non-increasing in γ and z.

We shall establish Original reference 2 (concretely, Theorem 2.8) using a proof by induction. The base case is straightforward and handled by the following lemma.

Lemma 2.2 (Base case)**.** *If a random tape adversarially robust algorithm* $\textsc{mif}(n, \ell)$ *uses at most* $\ell^2/(16n \ln 2)$ *bits of space and aborts with probability* $\le \frac{1}{2}$*, then it makes a mistake with probability* $\ge \frac{1}{4}$*. Equivalently,*

$$\Delta(n, \ell, \gamma, z) \ge \frac{1}{4} \mathbb{1}_{z \le \ell^2/(16n \ln 2)}\, \mathbb{1}_{\gamma \le 1/2}\,. \tag{1}$$

Proof. Suppose that $\gamma \le \frac{1}{2}$. Let $\mathcal{A} \in \textsc{Algs}(n, \ell, \gamma, z)$ be an algorithm with mistake probability $\delta \le \frac{1}{4}$. Blurring the distinction between aborts and mistakes, and applying the space lower bound for random *oracle* algorithms (Theorem 1.2), we obtain

$$z \ge \frac{\ell^2}{4n \ln 2} + \log(1 - \gamma - \delta) \ge \frac{\ell^2}{4n \ln 2} - 2 > \frac{\ell^2}{16n \ln 2}\,.$$

The latter inequality holds because $z \ge 1$ (trivially) and we have $\max(1, x-2) > x/4$. Taking the contrapositive, if $z \le \ell^2/(16n \ln 2)$, then $\delta > 1/4$. This proves eq. (1). □

2.2 The Induction Step

The induction step consists of a reduction, using an adaptive adversary described in Listing 1, to prove a lower bound on the mistake probability. This is formalized in the next lemma and the rest of Section 2.2 is devoted to its proof.

Lemma 2.3 (Induction lemma)**.** *Let $1 \le \ell < n$ and z be integers. Define, matching definitions in Listing 1,*

$$w := 2\left\lfloor 32\frac{zn}{\ell} \right\rfloor \qquad \text{and} \qquad t := \left\lfloor \frac{\ell}{64z} \right\rfloor . \tag{2}$$

If $z \ge 8$ and $t < w$, then:

$$\Delta\Big(n, \ell, \frac{1}{2}, z\Big) \ge \min\left(\frac{\ell}{2^7 nk}, \frac{1}{4}\Delta\Big(w, t, \frac{1}{2}, z\Big)\right) . \tag{3}$$

The Initial Random Prefix. The adversary begins by sending $\mathcal{A}$ a uniformly random sequence $X \in_R \textsc{seqs}([n], q)$, i.e., a sorted sequence of q distinct elements of $[n]$; we'll eventually use $q = \lceil \ell/2 \rceil$. Let F be the *random function* where, for $x \in \textsc{seqs}([n], q)$, $F(x)$ is the random state reached by $\mathcal{A}$ upon processing x, starting at its initial state; note that F is determined by the transition function T of $\mathcal{A}$ (see Section 0.1). Let Σ be the set of states of $\mathcal{A}$, so $|\Sigma| = 2^z$. For each $\sigma \in \Sigma$, define

$$H_\sigma := \left\{ i \in [n] : \Pr[i \in X \mid F(X) = \sigma] \le \frac{q}{4n} \right\} , \tag{4}$$

which we can think of as the set of inputs that are "unlikely" to have been seen given that $\mathcal{A}$ has reached σ. A key part of our proof of Theorem 2.3 is the following lemma, which says that if Σ is small, then $\mathcal{A}$ doesn't have enough space to mark too many inputs as unlikely, so $H_{F(X)}$ is likely to be small. The lemma can be seen as a smoothed variant of the communication lower bound for AVOID.

Lemma 2.4. *Let Σ be a set with $|\Sigma| \le 2^z$, let $1 \le q \le n$ be integers, let F be a random function that maps each sequence in $\textsc{seqs}([n], q)$ to a random element of Σ, and let $X \in_R \textsc{seqs}([n], q)$, chosen independently of F. For each $\sigma \in \Sigma$, define H_σ as in eq. (4). Then, for all $\alpha \in (0, 1)$,*

$$\Pr\left[\left|H_{F(X)}\right| \ge \hat{w}\right] \le \alpha \qquad \text{where} \qquad \hat{w} := \left\lceil \frac{2\ln 2}{1 - \ln 2} \frac{z + 1 + \log\frac{1}{\alpha}}{q}\, n \right\rceil .$$

Proof. Consider a specific $\sigma \in \Sigma$. By linearity of expectation:

$$\mathbb{E}\left[\sum_{i \in H_\sigma} \mathbb{1}_{i \in X} \;\middle|\; F(X) = \sigma \right] = \sum_{i \in H_\sigma} \Pr\left[i \in X \mid F(X) = \sigma \right] \le \frac{q}{4n} \left| H_\sigma \right| .$$

Then by Markov's inequality,

$$\Pr\left[\sum_{i \in H_\sigma} \mathbb{1}_{i \in X} \ge 2 \cdot \frac{q}{4n} \left| H_\sigma \right| \;\middle|\; F(X) = \sigma \right] \le \frac{1}{2}$$

$$\text{which implies} \qquad \Pr\left[\sum_{i \in H_\sigma} \mathbb{1}_{i \in X} \le \frac{q}{2n} \left| H_\sigma \right| \;\middle|\; F(X) = \sigma \right] \ge \frac{1}{2} .$$

Since X is drawn uniformly at random from $\textsc{seqs}([n], q)$, the random variables $\{\mathbb{1}_{i\in X}\}_{i\in[n]}$ are negatively associated, with $\mathbb{E}\mathbb{1}_{i\in X} = q/n$ for each $i \in [n]$. For any set $A \subseteq [n]$, we use the multiplicative Chernoff bound (Theorem 1.4) to bound the probability that X's overlap with A is much smaller than the expected value:

$$\Pr\left[\sum_{i\in A} \mathbb{1}_{i\in X} \le \left(1-\frac{1}{2}\right)\cdot\frac{q}{n}|A|\right] \le \left(\frac{e^{-1/2}}{(1/2)^{1/2}}\right)^{(q/n)|A|} = \exp\left(-\frac{1}{2}(1-\ln 2)\frac{q}{n}|A|\right).$$

We now bound

$$\Pr\left[F\left(X\right)=\sigma\right] \le \frac{\Pr\left[\sum_{i\in H_\sigma}\mathbb{1}_{i\in X} \le \frac{q}{2n}|H_\sigma|\right]}{\Pr\left[\sum_{i\in H_\sigma}\mathbb{1}_{i\in X} \le \frac{q}{2n}|H_\sigma| \;\middle|\; F(X)=\sigma\right]} \le 2\exp\left(-\frac{1}{2}(1-\ln 2)\frac{q}{n}|H_\sigma|\right).$$

Finally, let $B = \{\sigma \in \Sigma : |H_\sigma| \ge \hat{w}\}$. Then

$$\begin{aligned}\Pr\left[\left|H_{F(X)}\right| \ge \hat{w}\right] &= \sum_{\sigma\in B}\Pr\left[F\left(X\right)=\sigma\right] \\ &\le 2^z\cdot 2\exp\left(-\frac{1}{2}(1-\ln 2)\frac{q}{n}\hat{w}\right) \\ &\le 2^z\cdot 2\exp\left(-\left(z+1+\log\frac{1}{\alpha}\right)\ln 2\right) \le 2^z\cdot 2\cdot 2^{-\left(z+1+\log\frac{1}{\alpha}\right)} \boxminus \alpha\,.\end{aligned}$$

The Recursive Phases and Win-Win Argument. Let ρ denote the random state in Σ reached by $\mathcal{A}$ upon processing the random sequence X. Having observed $\mathcal{A}$'s outputs (transcript) in response to X, the adversary can compute $\mathcal{D}$, the distribution of ρ conditioned on this transcript. The adversary has $\lfloor \ell/2 \rfloor$ more items to send to $\mathcal{A}$ and it organizes these into phases of t items each (see eq. (2)). Recall, from the discussion in Original section 2.2, that the adversary's goal is to identify a suitable sub-universe $W \subseteq [n]$ so that, in some phase, $\mathcal{A}$ can be seen as solving $\textsc{mif}(|W|, t)$ in this sub-universe. As it chooses a suitable input sequence for each phase, the adversary maintains the following objects to guide the choice:

- the evolving transcript of $\mathcal{A}$ given the inputs chosen so far;
- the corresponding distribution $\mathcal{D} \in \triangle[\Sigma]$;
- a set $Q \subseteq \Sigma$ where, for each $\sigma \in Q$, the set H_σ will be useful for determining W.

If the current set Q leads to a suitable W, the adversary's strategy for the next phase is to recursively run an optimal sub-adversary for $\textsc{mif}(|W|, t)$. If not, then (we shall show that) over the next phase, the adversary will be able to significantly shrink the set Q.

To make these ideas more concrete, we introduce some terminology below; these terms show up in Listing 1, which spells out the adversary's actions precisely.

Definition 2.5 (Divisive output sequence, Splitting adversary)**.** Let $\mathcal{A}$, Σ, and H_σ be as above and let $Q \subseteq \Sigma$. A sequence (of "outputs") $y \in [n]^t$ is said to be $\textsc{divisive}$ for Q if $|\{\sigma \in Q : y \subseteq H_\sigma\}| \le \frac{1}{2}|Q|$.

Say Υ is a t-length deterministic adversary, i.e., a function $\Upsilon\colon [n]^{\le t-1} \to [n]$. For each $\sigma \in \Sigma$, let $\textsc{Outs}(\sigma, \Upsilon)$ be the random variable in $[n]^t \cup \{\perp\}$ that gives the output if we run $\mathcal{A}$, starting at state σ, against the adversary Υ,[6] with $\perp$ indicating that $\mathcal{A}$ aborts. We say that Υ is α-SPLITTING for Q with respect to a distribution $\mathcal{D} \in \triangle[\Sigma]$ if

$$\Pr_{S\sim\mathcal{D}}[\textsc{Outs}(S, \Upsilon) \text{ is divisive for } Q] \ge \alpha\,,$$

with the convention that the value $\perp$ *is* divisive.[7]

Notice that, knowing $\mathcal{D}$ and Q, the adversary can determine whether a given Υ is α-splitting. This is implicit in Algorithm 1 of Listing 1.

Listing 1 An adversary for a random tape $\textsc{mif}(n, \ell)$ algorithm.

ADVERSARY (against algorithm $\mathcal{A}$)

$w \leftarrow 2\lfloor 32zn/\ell \rfloor$; $h_{\max} \leftarrow 32z$; $t \leftarrow \lfloor \ell/(2h_{\max}) \rfloor$
$X \leftarrow$ a uniformly random sequence in $\textsc{seqs}([n], \lceil \ell/2 \rceil)$.
send X to $\mathcal{A}$ and record the transcript of outputs
compute H_σ for each state σ using eq. (4), with $q = \lceil \ell/2 \rceil$
$Q_0 \leftarrow \{\sigma \in \Sigma : |H_\sigma| \le \frac{1}{2}w\}$
for h in $1, \ldots, h_{\max}$ **do**
    $\mathcal{D} \leftarrow$ distribution over Σ conditioned on the cumulative transcript so far
    if $\exists$ a t-length deterministic adversary Υ that is $\frac{1}{2}$-splitting for Q_{h-1} w.r.t. $\mathcal{D}$ **then**
        run Υ against $\mathcal{A}$ and gather the transcript of outputs $y \in [n]^t$
        $Q_h \leftarrow \{\sigma \in Q_{h-1} : y \subseteq H_\sigma\}$ ▷ *have a $\ge \frac{1}{2}$ chance that $|Q_h| \le \frac{1}{2}|Q_{h-1}|$*
        if $Q_h = \varnothing$ **then fail**
    else
        $W \leftarrow \{i \in [n] : |\{\sigma \in Q_{h-1} : i \in H_\sigma\}| \ge \frac{1}{2}|Q_{h-1}|\}$ ▷ *will show that $|W| \le w$*
        $W' \leftarrow W$ plus $w - |W|$ padding elements
        define algorithm $\mathcal{B}$ to behave like $\mathcal{A}$ conditioned on the transcript of inputs and outputs so far
        modify $\mathcal{B}$, changing every output outside W' to $\perp$ ▷ *thus $\mathcal{B}$ will abort in these cases*
        let Ξ be an adversary using inputs from W', maximizing the probability that $\mathcal{B}$ makes a mistake
                ▷ *can be computed using brute-force search*
        run adversary Ξ, sending t inputs in W'
        return ▷ *succeeded in causing $\mathcal{A}$ to have a high enough error probability*
fail

We proceed to prove the induction lemma.

[6] This means that if, after processing a few inputs, the algorithm has output sequence $v \in [n]^\star$, its next input will be $\Upsilon(v)$.

[7] The proof can also be made to work if one assumes $\perp$ is not divisive, but gives a weaker and more complicated result.

Proof of Theorem 2.3. To prove the lower bound in eq. (3), we show that when the adversary in Listing 1 is run against a z-bit random-tape algorithm $\mathcal{A}$ for $\textsc{mif}(n, \ell)$ which has $\leq \frac{1}{2}$ worst-case probability of aborting, the probability that $\mathcal{A}$ makes a mistake is at least the right hand side of eq. (3). Note that the adversary feeds at most $\lceil \ell/2 \rceil + th_{\max} = \lceil \ell/2 \rceil + \lfloor \ell/(2h_{\max}) \rfloor h_{\max} \leq \lceil \ell/2 \rceil + \lfloor \ell/2 \rfloor = \ell$ inputs to $\mathcal{A}$.

Consider a run of the adversary against $\mathcal{A}$. This is a random process, with some of the randomness coming from the adversary's choices (Algorithm 1) and some coming from $\mathcal{A}$'s internal randomness. Let ρ be the state of $\mathcal{A}$ after X is sent. We now define a number of events, as follows.

- $B_{\textsc{unsafe}}$ occurs if $\mathcal{A}$ produces an output in $[n] \setminus H_\rho$.
- $B_{\textsc{big}}$ occurs if the state ρ has $|H_\rho| > \frac{1}{2}w$.
- $B_{\textsc{empty}}$ occurs if the adversary fails at Algorithm 1.
- $B_{\textsc{timeout}}$ occurs if the adversary fails at Algorithm 1.
- $B_{\textsc{abort}}$ occurs if $\mathcal{A}$ aborts *before* the adversary reaches Algorithm 1.
- $R_{\textsc{error}}$ occurs if $\mathcal{A}$ makes a mistake *while* the adversary is executing Algorithm 1.

We will consider each of the events listed above.

(Event $B_{\textsc{unsafe}}$) If $B_{\textsc{unsafe}}$ occurs, then some $i \in [n] \setminus H_\rho$ is output and, in view of eq. (4), the set X from Algorithm 1 has the property that

$$\Pr[i \in X \mid \mathcal{A} \text{ reaches } \rho] \geq \frac{\lceil \ell/2 \rceil}{4n} \geq \frac{\ell}{8n}\,.$$

Consequently, the probability that $\mathcal{A}$ makes a mistake by producing an output from X (which would be a non-missing item) is $\geq \Pr[B_{\textsc{unsafe}}] \cdot \ell/(8n)$.

If $\Pr[B_{\textsc{unsafe}}] > 1/(16)$, we then have $\Delta(n, \ell, \frac{1}{2}, z) \geq \ell/(2^7 n)$, which implies eq. (3) leaving nothing more to prove. Therefore, for the rest of this proof we will consider the case in which

$$\Pr[B_{\textsc{unsafe}}] \leq \frac{1}{16}\,. \tag{5}$$

(Event $B_{\textsc{big}}$) We apply Theorem 2.4 with $q = \lceil \ell/2 \rceil$, $\alpha = 1/16$, F being the random function that maps each $x \in \textsc{seqs}([n], q)$ to the random state reached by $\mathcal{A}$ upon processing x, starting at its initial state, and

$$\begin{aligned}
\hat{w} = \left\lceil \frac{2\ln 2}{1-\ln 2} \frac{z+1+\log 16}{\lceil \ell/2 \rceil}\, n \right\rceil &\leq 1 + \left\lfloor \frac{2\ln 2}{1-\ln 2} \frac{z+1+\log 16}{\lceil \ell/2 \rceil}\, n \right\rfloor \\
&\leq 1 + \left\lfloor \frac{8\ln 2}{1-\ln 2} \frac{z+1+\log 16}{\ell}\, n \right\rfloor \\
&\leq 1 + \left\lfloor 19 \frac{z+5}{\ell} n \right\rfloor \leq 1 + \left\lfloor 32 \frac{zn}{\ell} \right\rfloor = \frac{1}{2}w + 1\,,
\end{aligned}$$

since $z \geq 8$. As $\frac{1}{2}w$ is an integer,

$$\Pr\left[B_{\textsc{big}}\right] = \Pr\left[|H_\rho| > \frac{1}{2}w\right] = \Pr\left[|H_\rho| \geq \frac{1}{2}w + 1\right] \leq \Pr\left[|H_\rho| \geq \hat{w}\right] \leq \frac{1}{16}\,. \tag{6}$$

(Event $B_{\textbf{abort}}$) There are exactly exactly three spots in Listing 1 where the algorithm can abort: Algorithm 1, when the adversary is feeding it inputs. By assumption, $\mathcal{A}$'s worst case probability of aborting, against *any* adversary, is at most $\frac{1}{2}$. Therefore, in particular,

$$\Pr[B_{\text{ABORT}}] \leq \frac{1}{2}\,. \tag{7}$$

When the algorithm does abort, we stop running the adversary, so B_{ABORT}, B_{EMPTY}, and B_{TIMEOUT} are mutually exclusive.

(Event $B_{\textbf{empty}}$) For this to happen, at some point we must have $Q_h = \varnothing$. At that point, the state ρ must not be in Q_h, in which case either $\rho \notin Q_0$ or ρ was filtered out of Q_h on Algorithm 1. By the definition of Q_0, $\rho \notin Q_0$ iff B_{BIG} holds. On the other hand, filtering ρ out of Q_h requires that the algorithm produce an output outside H_ρ, which can only happen if either B_{UNSAFE} or B_{ABORT} occurs. Since B_{ABORT} is mutually exclusive with B_{EMPTY}, we conclude that

$$B_{\text{EMPTY}} \Rightarrow B_{\text{UNSAFE}} \vee B_{\text{BIG}}\,. \tag{8}$$

(Event $B_{\textbf{timeout}}$) We bound the probability that the adversary will fail using Algorithm 1. For this to happen, the adversary must have picked $h_{\max}$ splitting adversaries, but fewer than $z+1$ of them must have produced a divisive output. (If there is a divisive output in round h, then $|Q_h| \leq \frac{1}{2}|Q_{h-1}|$; if not, then $|Q_h| \leq |Q_{h-1}|$. Thus with $z+1$ divisive outputs, $|Q_{h_{\max}}| \leq |Q_0|/2^{z+1} \leq |\Sigma|/2^{z+1} \leq \frac{1}{2} < 1$, in which case the adversary would have failed at Algorithm 1 instead.)

For each $h \in [h_{\max}]$, let X_h be the $\{0,1\}$ indicator random variable for the event that a divisive output is found in the hth step. (If the hth step did not occur or no splitting adversary was found, set $X_h = 1$.[8]) Since in the hth step, a splitting adversary for the distribution for the current state of the algorithm, conditioned on the transcript so far, is chosen, then $\mathbb{E}[X_h \mid X_1, \ldots, X_{h-1}] \geq 1/2$. Applying Theorem 1.3 gives:

$$\begin{aligned}
\Pr\Big[\sum_{h\in[h_{\max}]} X_h < z+1\Big] &= \Pr\left[\sum_{h\in[h_{\max}]} X_h \leq \left(1-\left(1-\frac{2z}{h_{\max}}\right)\right)\frac{h_{\max}}{2}\right] \\
&\leq \exp\left(-\frac{1}{2}\left(1-\frac{2z}{h_{\max}}\right)^2\frac{h_{\max}}{2}\right) \\
&\leq \exp\left(-\frac{1}{8}\frac{h_{\max}}{2}\right) && \text{since } h_{\max} = 32z \geq 4z \\
&\leq \frac{1}{8}\,. && \text{since } h_{\max} \geq 16\ln 8
\end{aligned}$$

Thus $\Pr[B_{\text{TIMEOUT}}] \leq 1/8$.

[8] Note that this definition accounts for the cases where the algorithm aborts: if it aborts on h, by Theorem 2.5 this is interpreted as divisive, and the following steps do not occur, so $X_h = X_{h+1} = \ldots, X_{h_{\max}} = 1$. Then the event $\{\sum X_h < z+1\}$ slightly overestimates the probability of B_{TIMEOUT}; it would be more accurate to have $\mathcal{A}$ aborting produce $X_h = \infty$.

Combining this with eqs. (5) to (8) gives:

$$\begin{aligned}\Pr[B_{\text{EMPTY}} \vee B_{\text{TIMEOUT}} \vee B_{\text{ABORT}}] &\le \Pr[B_{\text{UNSAFE}} \vee B_{\text{BIG}} \vee B_{\text{TIMEOUT}} \vee B_{\text{ABORT}}] \\ &\le \Pr[B_{\text{UNSAFE}}] + \Pr[B_{\text{BIG}}] + \Pr[B_{\text{TIMEOUT}}] + \Pr[B_{\text{ABORT}}] \\ &\le \frac{1}{16} + \frac{1}{16} + \frac{1}{8} + \frac{1}{2} = \frac{3}{4}\,. \end{aligned} \tag{9}$$

(Event R_{error}) Let E be the event that the adversary executes Algorithm 1. Notice that R_{ERROR} can only occur when E occurs. Note also that

$$E = \neg B_{\text{EMPTY}} \wedge \neg B_{\text{TIMEOUT}} \wedge \neg B_{\text{ABORT}}\,,$$

so eq. (9) implies that

$$\Pr[E] \ge \frac{1}{4}\,. \tag{10}$$

Suppose that E does occur. We now make two claims: (a) that $|W| \le w$, and (b) that $\mathcal{B} \in \textsc{Algs}(w, t, \frac{1}{2}, z)$. For the first claim, note that by Algorithm 1, the set Q_0 only contains states $\sigma \in \Sigma$ with $|H_\sigma| \le \frac{1}{2}w$. The same bound on $|H_\sigma|$ holds for all $\sigma \in Q_{h-1}$ because $Q_{h-1} \subseteq Q_0$, thanks to Algorithm 1. By the definition of W (Algorithm 1),

$$\begin{aligned}|W| &= \left|\left\{i \in [n] : \frac{|\{\sigma \in Q_{h-1} : i \in H_\sigma\}|}{|Q_{h-1}|} \ge \frac{1}{2}\right\}\right| \\ &\le \sum_{i \in [n]} \frac{2|\{\sigma \in Q_{h-1} : i \in H_\sigma\}|}{|Q_{h-1}|} \\ &= \frac{2}{|Q_{h-1}|} \sum_{\sigma \in Q_{h-1}} |H_\sigma| \le \frac{2}{|Q_{h-1}|} \cdot |Q_{h-1}| \cdot \frac{1}{2}w = w\,,\end{aligned}$$

which proves claim (a).

For the second claim, consider the sub-algorithm $\mathcal{B}'$ defined as $\mathcal{B}$ just before the modification at Algorithm 1. Then, for a particular adversary Υ, running Υ against $\mathcal{B}$ causes an abort exactly when running Υ against $\mathcal{B}'$ causes either an abort or an output outside W'. In the iteration of the **for** loop that caused E, since we reached the **else** branch, there was no deterministic splitting adversary with respect to $\mathcal{D}$. Thus for *any* deterministic adversary Υ, by Theorem 2.5,

$$\Pr_{\hat{\sigma} \sim \mathcal{D}}[\textsc{Outs}(\hat{\sigma}, \Upsilon) \text{ is divisive for } Q_{h-1}] \le \frac{1}{2}\,.$$

Let y be a realization of the random variable $\textsc{Outs}(\hat{\sigma}, \Upsilon)$; note that $y \in [n]^t \cup \{\perp\}$. If a given y is not divisive, then $y \in [n]^t$ and, for each $i \in y$,

$$|\{\sigma \in Q_{h-1} : i \in H_\sigma\}| \ge |\{\sigma \in Q_{h-1} : y \subseteq H_\sigma\}| \ge \frac{1}{2}|Q_{h-1}|\,,$$

which implies that $i \in W$. Thus in fact $y \subseteq W$. It follows that the probability that running Υ against $\mathcal{B}'$ causes an abort or an output outside W (which is a subset of W') is at most $\frac{1}{2}$. Viewing $\mathcal{B}$ as a random-tape MIF algorithm handling

input streams of length at most t, with items from W', in the terminology of Theorem 2.1, we have $\mathcal{B} \in \textsc{Algs}(w, t, \frac{1}{2}, z)$. This proves claim (b).

Since Ξ is picked to maximize the probability of $\mathcal{B}$ making a mistake, we have $\Pr[R_{\text{ERROR}} \mid E] \geq \Delta(w, t, \frac{1}{2}, z)$. Using eq. (10), we obtain

$$\Pr[R_{\text{ERROR}}] \geq \Pr[E]\,\Delta\left(w, t, \frac{1}{2}, z\right) \geq \frac{1}{4}\Delta\left(w, t, \frac{1}{2}, z\right) .$$

Combining this lower bound with the lower bound for the case where $\Pr[B_{\text{UNSAFE}}] > \frac{1}{16}$ (considered just before eq. (5)), we obtain

$$\Delta\left(n, \ell, \frac{1}{2}, z\right) \geq \min\left(\frac{\ell}{2^7 n},\ \frac{1}{4}\Delta\left(w, t, \frac{1}{2}, z\right)\right) . \qquad \square$$

2.3 Calculating the Lower Bound

Lemma 2.6. *Let $1 \leq \ell < n$. For any integer $k \geq 1$, say that z is an integer satisfying $z \leq \frac{1}{256}\ell^{1/k}$. Then:*

$$\Delta(n, \ell, 0, z) > \min\left(\frac{\ell}{2^7 n}, \frac{1}{4^k}\mathbb{1}_{z \leq L}\right) \qquad \textit{where} \qquad L = \frac{1}{64}\left(\frac{\ell^{k+1}}{n}\right)^{\frac{2}{k^2+3k-2}} . \tag{11}$$

Consequently, algorithms for MIF with $\leq \min(\frac{\ell}{2^7 n}, 4^{-k})$ error require $> L$ bits of space.

Proof of Theorem 2.6. Let $n_1 = n$ and $\ell_1 = \ell$, and for $i = 2, \ldots, k$, set $n_i = 2\left\lfloor 32\frac{z n_{i-1}}{\ell_{i-1}}\right\rfloor$ and $\ell_i = \left\lfloor \frac{\ell_{i-1}}{64z}\right\rfloor$. This matches the definitions used in Theorem 2.3. As we have been promised that $z \leq \frac{1}{256}\ell^{1/k}$, we have in particular that:

$$(256z)^k \leq \ell \qquad \text{which implies} \qquad \ell_1 \geq \cdots \geq \ell_k \geq 256z\,.$$

By Theorem 1.5, we only need to consider the case $z \geq \log(\ell + 1)$, as otherwise $\Delta(n, \ell, 0, z) = 1$. Thus, if $k \geq 2$, we have:

$$z \geq \log(\ell + 1) \geq k \log(256z) \geq k \log(256) = 16 \geq 8\,.$$

(If $k = 1$, then eq. (11) follows immediately from Theorem 2.2.)

Since $\Delta(n, \ell, 0, z) \geq \Delta(n, \ell, 1/2, z)$, it suffices to lower bound the case where algorithms are permitted up to $1/2$ abort probability. We will lower bound $\Delta(n, \ell, 1/2, z)$ by recursively applying Theorem 2.3 $k - 1$ times, and then applying Theorem 2.2. This yields:

$$\begin{aligned}
\Delta(n, \ell, 1/2, z) &\geq \min\left(\frac{\ell_1}{2^7 n_1}, \frac{1}{4}\Delta(n_1, \ell_1, 1/2, z)\right) \\
&\geq \min\left(\frac{\ell_1}{2^7 n_1}, \frac{1}{4}\min\left(\frac{\ell_2}{2^7 n_2}, \ldots \frac{1}{4}\min\left(\frac{\ell_{k-1}}{2^7 n_{k-1}}, \frac{1}{4}\mathbb{1}_{z \leq \ell_k^2/(16 n_k \ln 2)}\right)\cdots\right)\right) \\
&\geq \min\left(\frac{\ell_1}{2^7 n_1}, \frac{1}{4}\frac{\ell_2}{2^7 n_2}, \ldots, \frac{1}{4^{k-1}}\frac{\ell_{k-1}}{2^7 n_{k-1}}, \frac{1}{4^k}\mathbb{1}_{z \leq \ell_k^2/(16 n_k \ln 2)}\right) .
\end{aligned}$$

Only the first and last terms of the minimum are significant, because the terms for $i = 2, \ldots, k-1$ are all dominated by the first term:

$$\frac{\ell_i}{n_i} = \frac{\left\lfloor \frac{\ell_{i-1}}{64z} \right\rfloor}{2\left\lfloor 32\frac{zn_{i-1}}{\ell_{i-1}} \right\rfloor} \geq \frac{1}{2}\frac{\ell_{i-1}}{64z}\frac{\ell_{i-1}}{64zn_{i-1}} \geq \frac{\ell_{i-1}}{2(64z)^2}\frac{\ell_{i-1}}{n_{i-1}} \geq 4\frac{\ell_{i-1}}{n_{i-1}},$$

where in the last step, we used the fact that $\ell_{i-1} \geq \ell_{k-2} \geq (256z)^2$. Thus:

$$\Delta(n, \ell, 0, z) \geq \min\left(\frac{\ell}{2^7 nk}, \frac{1}{4^k}\mathbb{1}_{z \leq \ell_k^2/(16n_k \ln 2)}\right).$$

We have almost proven eq. (11). It remains to lower bound $\mathbb{1}_{z \leq \ell_k^2/(16n_k \ln 2)}$ by $\mathbb{1}_{z \leq L}$ for some L. We do so by proving $z > \ell_k^2/(16n_k \ln 2)$ implies $z > L$. As a consequence of the definitions, we have:

$$\ell_i = \left\lfloor \frac{\ell_{i-1}}{64z} \right\rfloor = \left\lfloor \frac{\left\lfloor \frac{\ell_{i-2}}{64z} \right\rfloor}{64z} \right\rfloor = \cdots = \left\lfloor \frac{\ell}{(64z)^{i-1}} \right\rfloor \qquad \text{and} \qquad n_i \leq 64\frac{zn_{i-1}}{\ell_{i-1}},$$

and thus:

$$\begin{aligned} z &> \frac{\ell_k^2}{16\ln 2}\frac{1}{n_k} \geq \frac{\ell_k^2}{16}\frac{\prod_{k=1}^{k-1}\ell_k}{(64z)^{k-1}n} \\ &\geq \frac{\ell^{k+1}}{16 \cdot (64z)^{2(k-1)+(k-2)+(k-3)+\cdots+1+0}(64z)^{k-1}n} \\ &= \frac{\ell^{k+1}}{16 \cdot (64z)^{(k^2+3k-4)/2}n}. \end{aligned}$$

Rearranging to put all the z terms on the left gives:

$$64z \cdot (64z)^{\frac{k^2+3k-4}{2}} \geq 64\frac{\ell^{k+1}}{16n},$$

which implies

$$z > \frac{1}{64}\left(\frac{64\ell^{k+1}}{16n}\right)^{\frac{2}{k^2+3k-2}} \geq \frac{1}{64}\left(\frac{\ell^{k+1}}{n}\right)^{\frac{2}{k^2+3k-2}}. \qquad \square$$

Now, say a random tape algorithm has error probability $\leq \frac{\ell}{2^7 n}$ against any adaptive adversary, and uses z bits of space. If $\ell \geq n^{2/3}$, it is easy to show that the optimal value of k with which to apply Theorem 2.6 is 1. Otherwise, for all k, we have two cases: if $z \leq \frac{1}{256}\ell^{1/k}$, then we can apply Theorem 2.6. By Theorem 1.5, $z\log(\ell+1) \geq 1$, which implies $(256)^k \leq \ell$ and thus $k \leq \left\lfloor \frac{1}{8}\log \ell \right\rfloor$. As the algorithm has

$$\text{error} \leq \frac{\ell}{2^7 n} \leq \min\left(\frac{\ell}{2^7 n}, \frac{1}{2^7 \ell^{1/2}}\right) \leq \min\left(\frac{\ell}{2^7 n}, \frac{1}{4^k}\right),$$

by Theorem 2.6 the algorithm's space usage must satisfy:

$$z \geq \frac{1}{64}\left(\frac{\ell^{k+1}}{n}\right)^{\frac{2}{k^2+3k-2}}.$$

Since we either have this lower bound on z, or $z > \frac{1}{256}\ell^{1/k}$, it follows that for any integer $k \geq 1$:

$$z \geq \frac{1}{256} \min\left(\ell^{1/k}, \left(\frac{\ell^{k+1}}{n}\right)^{\frac{2}{k^2+3k-2}}\right).$$

We can take the maximum over all values of k to obtain:

$$z \geq \frac{1}{256} \max_{k\in\mathbb{N}} \min\left(\ell^{1/k}, \left(\frac{\ell^{k+1}}{n}\right)^{\frac{2}{k^2+3k-2}}\right).$$

The right hand side can be simplified with the following lemma, whose proof is mostly calculation and is deferred to Section 2.4:

Lemma 2.7.

$$\max_{k\in\mathbb{N}} \min\left(\ell^{1/k}, \left(\frac{\ell^{k+1}}{n}\right)^{\frac{2}{k^2+3k-2}}\right) \geq \max_{k\in\mathbb{N}} \left(\frac{\ell^{k+1}}{n}\right)^{\frac{2}{k^2+3k-2}} \geq \ell^{\frac{15\log\ell}{32\log n}} \tag{12}$$

Summarizing, we obtain the following theorem.

Theorem 2.8. *Random tape δ-error adversarially robust algorithms for* MIF(n,ℓ) *require*

$$\Omega\left(\max_{k\in\mathbb{N}} \left(\frac{\ell^{k+1}}{n}\right)^{\frac{2}{k^2+3k-2}}\right) = \Omega\left(\ell^{\frac{15}{32}\log_n \ell}\right)$$

bits of space, for $\delta \leq \ell/(2^7 n)$. □

2.4 Mechanical Proofs for Section 2

Proof of Theorem 2.7. Let x be the left hand side of Eq. 12. The left branch of the $\min(\cdot,\cdot)$ terms in Eq. 12 is actually unnecessary. For any integer $\lambda \geq 2$, say that

$$\left(\frac{\ell^{\lambda+1}}{n}\right)^{\frac{2}{\lambda^2+3\lambda-2}} = \max_{k\in\mathbb{N}} \left(\frac{\ell^{k+1}}{n}\right)^{\frac{2}{k^2+3k-2}}.$$

Then in particular,

$$\left(\frac{\ell^{\lambda+1}}{n}\right)^{\frac{2}{\lambda^2+3\lambda-2}} \geq \left(\frac{\ell^{(\lambda-1)+1}}{n}\right)^{\frac{2}{(\lambda-1)^2+3(\lambda-1)-2}} \geq \left(\frac{\ell^{\lambda}}{n}\right)^{\frac{2}{\lambda^2+\lambda-4}},$$

which implies

$$n^{2\lambda+2} = n^{(\lambda^2+3\lambda-2)-(\lambda^2+\lambda-4)} \geq \ell^{\lambda\cdot(\lambda^2+3\lambda-2)-(\lambda+1)\cdot(\lambda^2+\lambda-4)} = \ell^{\lambda^2+\lambda+4},$$

hence we have $\ell \leq n^{\frac{2\lambda+2}{\lambda^2+\lambda+4}}$.

On the other hand, we have

$$\ell^{1/\lambda} \geq \left(\frac{\ell^{\lambda+1}}{n}\right)^{\frac{2}{\lambda^2+3\lambda-2}} \qquad \Longleftrightarrow \qquad n^{\lambda} \leq \ell^{(\lambda+1)\lambda-\frac{\lambda^2+3\lambda-2}{2}} = \ell^{\frac{\lambda^2-\lambda+2}{2}},$$

so the left branch of the $\min(\cdot,\cdot)$ in Eq. 12 is only smaller when $\ell \geq n^{\frac{2\lambda}{\lambda^2-\lambda+2}}$. As

$$\frac{2\lambda+2}{\lambda^2+\lambda+4} \leq \frac{2\lambda}{\lambda^2-\lambda+2},$$

for all $\lambda \geq 1$, it follows that the left branch of the $\min(\cdot,\cdot)$ in Eq. 12 is only smaller than the right when the entire term is not the maximum. Thus

$$x \geq \max_{k\in\mathbb{N}} \left(\frac{\ell^{k+1}}{n}\right)^{\frac{2}{k^2+3k-2}}.$$

To get a looser but more easily comprehensible lower bound, we note that $\max_{k\in\mathbb{N}} \log\left(\frac{\ell^{k+1}}{n}\right)^{\frac{2}{k^2+3k-2}}$ is piecewise linear and convex in $\log \ell$. Consequently, we can lower bound it using the convex function $C\frac{(\log \ell)^2}{\log n}$, where C is the maximum value which satisfies the inequality at all "corner points" of $\max_{k\in\mathbb{N}} \log\left(\frac{\ell^{k+1}}{n}\right)^{\frac{2}{k^2+3k-2}}$. These corner points occur precisely at values of $\log \ell$ where, for some $k \geq 2$, we have:

$$\left(\frac{\ell^{k+1}}{n}\right)^{\frac{2}{k^2+3k-2}} = \left(\frac{\ell^{(k-1)+1}}{n}\right)^{\frac{2}{(k-1)^2+3(k-1)-2}}.$$

Rearranging this gives:

$$\frac{\log n}{\log \ell} = \frac{(k+1)((k-1)^2+3(k-1)-2)-(k-1+1)(k^2+3k-2)}{((k-1)^2+3(k-1)-2)-(k^2+3k-2)} = \frac{k^2+k+4}{2k+2}.$$

so the corners occur at $\ell = n^{\frac{2k+2}{k^2+k+4}}$; and at such ℓ, we have

$$\begin{aligned}
\log\left(\frac{\ell^{k+1}}{n}\right)^{\frac{2}{k^2+3k-2}} &= \left(\frac{2}{k^2+3k-2}\cdot((k+1)\frac{2k+2}{k^2+k+4}-1)\right)\log n = \frac{2}{k^2+k+4}\log n \\
&= \frac{(\log \ell)^2}{\log n}\frac{2}{k^2+k+4}\left(\frac{\log n}{\log \ell}\right)^2 = \frac{(\log \ell)^2}{\log n}\frac{2}{k^2+k+4}\left(\frac{k^2+k+4}{2k+2}\right)^2 \\
&= \frac{1}{2}\frac{(\log \ell)^2}{\log n}\frac{k^2+k+4}{(k+1)^2} \geq \frac{15}{32}\frac{(\log \ell)^2}{\log n}.
\end{aligned}$$

The function $\frac{k^2+k+4}{(k+1)^2}$ has derivative $\frac{k-7}{(k+1)^3}$ and is minimized exactly at $k=7$, where it has value $\frac{15}{16}$. Consequently, the value $C = \frac{15}{32}$ is the best possible. □

References

[ABJ+22] Miklós Ajtai, Vladimir Braverman, T.S. Jayram, Sandeep Silwal, Alec Sun, David P. Woodruff, and Samson Zhou. The white-box adversarial data stream model. In *Proc. 41st ACM Symposium on Principles of Database Systems*, pages 15–27, 2022.

[ACGS23] Sepehr Assadi, Amit Chakrabarti, Prantar Ghosh, and Manuel Stoeckl. Coloring in graph streams via deterministic and adversarially robust algorithms. In *Proc. 42nd ACM Symposium on Principles of Database Systems*, pages 141–153, 2023.

[BEEO22] Omri Ben-Eliezer, Talya Eden, and Krzysztof Onak. Adversarially robust streaming via dense-sparse trade-offs. In *Symposium on Simplicity in Algorithms (SOSA)*, pages 214–227, 2022.

[BJWY20] Omri Ben-Eliezer, Rajesh Jayaram, David P. Woodruff, and Eylon Yogev. A framework for adversarially robust streaming algorithms. In *Proc. 39th ACM Symposium on Principles of Database Systems*, page 63–80, 2020.

[BKKS23] Vladimir Braverman, Robert Krauthgamer, Aditya Krishnan, and Shay Sapir. Lower bounds for pseudo-deterministic counting in a stream. *arXiv preprint arXiv:2303.16287*, 2023.

[BR93] Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In *Proc. 1st ACM Conference on Computer and Communications Security*, pages 62–73, 1993.

[BY20] Omri Ben-Eliezer and Eylon Yogev. The adversarial robustness of sampling. In *Proc. 39th ACM Symposium on Principles of Database Systems*, pages 49–62. ACM, 2020.

[CGS22] Amit Chakrabarti, Prantar Ghosh, and Manuel Stoeckl. Adversarially robust coloring for graph streams. In *Proc. 13th Conference on Innovations in Theoretical Computer Science*, pages 37:1–37:23, 2022.

[Fei19] Uriel Feige. A randomized strategy in the mirror game. *arXiv preprint arXiv:1901.07809*, 2019.

[GGMW20] Shafi Goldwasser, Ofer Grossman, Sidhanth Mohanty, and David P. Woodruff. Pseudo-Deterministic Streaming. In *Proc. 20th Conference on Innovations in Theoretical Computer Science*, volume 151, pages 79:1–79:25, 2020.

[GGS23] Ofer Grossman, Meghal Gupta, and Mark Sellke. Tight space lower bound for pseudo-deterministic approximate counting. *arXiv preprint arXiv:2304.01438*, 2023.

[GS18] Sumegha Garg and Jon Schneider. The Space Complexity of Mirror Games. In *Proc. 10th Conference on Innovations in Theoretical Computer Science*, pages 36:1–36:14, 2018.

[HKM+20] Avinatan Hassidim, Haim Kaplan, Yishay Mansour, Yossi Matias, and Uri Stemmer. Adversarially robust streaming algorithms via differential privacy. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020.

[HW13] Moritz Hardt and David P. Woodruff. How robust are linear sketches to adaptive inputs? In *Proc. 45th Annual ACM Symposium on the Theory of Computing*, pages 121–130, 2013.

[Ind06] Piotr Indyk. Stable distributions, pseudorandom generators, embeddings, and data stream computation. *J. ACM*, 53(3):307–323, 2006.

[JP83] Kumar Joag-Dev and Frank Proschan. Negative association of random variables, with applications. *Ann. Stat.*, 11(1):286–295, 1983.

[JW23] Rajesh Jayaram and David P Woodruff. Towards optimal moment estimation in streaming and distributed models. *ACM Trans. Alg.*, 19(3):1–35, 2023.

[KMNS21] Haim Kaplan, Yishay Mansour, Kobbi Nissim, and Uri Stemmer. Separating adaptive streaming from oblivious streaming using the bounded storage model. In *Advances in Cryptology - CRYPTO 2021 - 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16-20, 2021, Proceedings, Part III*, volume 12827 of *Lecture Notes in Computer Science*, pages 94–121. Springer, 2021.

[MN22a] Roey Magen and Moni Naor. Mirror games against an open book player. In *11th International Conference on Fun with Algorithms (FUN 2022)*, volume 226, pages 20:1–20:12, 2022.

[MN22b] Boaz Menuhin and Moni Naor. Keep that card in mind: Card guessing with limited memory. In *Proc. 13th Conference on Innovations in Theoretical Computer Science*, pages 107:1–107:28, 2022.

[New91] Ilan Newman. Private vs. common random bits in communication complexity. *Inform. Process. Lett.*, 39(2):67–71, 1991.

[Nis90] Noam Nisan. Pseudorandom generators for space-bounded computation. In *Proc. 22nd Annual ACM Symposium on the Theory of Computing*, pages 204–212, 1990.

[Nis93] Noam Nisan. On read once vs. multiple access to randomness in logspace. *Theoretical Computer Science*, 107(1):135–144, 1993.

[Sto23] Manuel Stoeckl. Streaming algorithms for the missing item finding problem. In *Proc. 34th Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 793–818, 2023. Full version at arXiv:2211.05170v1.

[WZ22] David P. Woodruff and Samson Zhou. Tight bounds for adversarially robust streams and sliding windows via difference estimators. In *Proc. 62nd Annual IEEE Symposium on Foundations of Computer Science*, pages 1183–1196, 2022.